

Ledger Core: Architecture & Trade-offs

Part 2 · In-memory account ledger core (TypeScript) · code references are to `src/ledger.ts`

1. Append-only at scale

What breaks first at 100×: the authorization path. Nothing is indexed; every read is a linear scan of one global array. An auth decision costs `ledgerBalance + activeHolds`, which is $O(N + A^2)$ over the whole bank, not one account. At 100× auth volume the A^2 term alone costs 10 000× as much per decision. It is also the only synchronous, deadline-bound path: the card network is waiting, and a timeout becomes a stand-in decision or a decline. The nightly batch fails second; it merely runs late.

Operation	Cost today	Called from
<code>ledgerBalance(account, day)</code>	$O(\text{all entries, all accounts})$	every auth, fee day, accrual and report cell
<code>activeHolds(account)</code>	$O(A^2)$: <code>authIds()</code> collects every auth ever, then <code>authState()</code> rescans the auth log per ID	every auth decision, every nightly snapshot
<code>trueUpFees / capitalizeInterest</code>	$O(D \times N)$ per account, from day 1 every night	nightly
Nightly snapshot history	$O(D \times N)$ per account, and stored	nightly

Check-then-hold (`availableBalance()` then `authLog.append`) is atomic only because JavaScript is single-threaded. With a second process, or any `await` in between, two auths can both pass against the same funds.

Unbounded state

- All four logs (`ledger`, `eventLog`, `authLog`, `accrualLog`) grow forever; `eventLog` also keeps rejected events. `diagnosticsLog` is unbounded too.
- **Nightly reports are the worst term:** each night stores every earlier day's balance again, so memory grows with $\text{nights}^2 \times \text{accounts}$.
- Fee and interest lookback run from `window.first` every night. There is no horizon after which history is closed.
- `AppendOnlyLog.all()` copies the full array on each call, and the engine calls it every night only to slice off the tail.

Cheapest structural change

Per-account, per-value-day balance buckets, maintained on append, plus a per-account open-holds counter updated on each auth transition. `append()` already sees every entry, so the extra work is $O(1)$. `ledgerBalance` becomes a prefix sum over at most D buckets for one account, and `activeHolds` becomes $O(1)$. The index is derived: the logs stay the only source of truth, and replay rebuilds it. That means no migration, no change to the audit model, and it fixes the authorization path first.

It defers the problem rather than solving it, because memory still grows with history. The next step is a **period close**: with the backdating horizon from §2 in place, write a checkpoint balance per account, stop lookback there, and move older log segments to cold storage. The logs are segmented, never edited. Reports store only tonight's changes, not the full history.

2. Value-dated entries in production (UAE-licensed bank)

A value date lets an entry change a balance that has **already been observed**, while the bank keeps acting on the new version. Every system that consumed the old balance is now wrong:

Area	Exposure created by a backdated entry
Customers	Statements and alerts already sent. A backdated debit creates an overdraft and a fee the customer could not have avoided. Under CBUAE consumer-protection expectations (fee transparency, fair treatment, complaints), this needs a written policy on when retroactive fees are charged or waived. Decisions such as a declined authorization cannot be restated at all.
VAT	Fee-based financial services are standard-rated at 5%; margin-based income such as interest is exempt. Every FEE_REFUND triggered by a correction needs a tax credit note against the original invoice, in the right VAT period. The engine books neither.
Accounting periods	A value date before month-end, booked after close, moves a period the GL has locked. Interest RESTATEMENT records are prior-period adjustments. Rule needed: post in the open period; use the value date only for customer interest and fee calculations.
Regulatory returns	CBUAE prudential returns and reserve-requirement figures use period-end or averaged balances. Backdating across a reporting cut-off silently changes a figure already submitted.
Islamic windows	"Daily interest" is profit allocation under a different contract. Restating profit shares needs Sharia-board-approved rules, not the same code path.
AML / CFT	Transaction monitoring usually runs over booking dates in daily batches. An entry value-dated into an already-scanned window can dodge velocity and threshold rules, and escape reporting to the UAE FIU (goAML) if rules key on value date.
Fraud	Value-dating is a classic insider lever: backdate a credit before a balance check to avoid a fee. Here, anyone who can submit an event may choose any past value date in the window; no authority, reason or approval is checked.

The one control before go-live: a backdating authority limit with maker-checker

Any entry valued earlier than the current business day needs a reason code, and posts automatically only within a small tolerance (for example the previous business day for scheme clearing). Anything older needs approval from a second user; the maker cannot approve. Anything before the last closed accounting period is **hard-rejected** and re-booked as a current-period adjustment. This one control caps GL period integrity, submitted returns, the monitoring gap and the fraud path, and it enables the period close in §1. In code it is a check in `apply()` beside the existing `FUTURE_VALUE_DATE` guard, plus maker, checker and `reasonCode` fields on the event.

3. Authorization lifecycle

The model has three statuses: APPROVED, DECLINED, SETTLED. Every way an authorization ends other than a matching settlement:

How it ends (where)	Real-world scenario	Mandated behaviour
1. Declined at decision: available after hold < 0 (applyAuthorization)	Insufficient funds (ISO 8583 response 51)	Answer within the scheme deadline; never re-judge. If a later correction removes the reason, raise a DECLINE_UNDERMINED case for ops and customer contact.
2. Never created: rejected at intake (DUPLICATE_AUTH, CURRENCY_MISMATCH, UNKNOWN_ACCOUNT, amount and window errors)	Network retransmission; foreign-currency purchase; closed or unknown card account	Duplicates must be idempotent : return the original response, not an error, which the network reads as a decline. Cross-currency: authorize the billing amount in account currency and disclose FX markup. Unknown account: decline, no hold.
3. Stays APPROVED forever: no expiry; finish() just stops	Merchant never captures: cancelled order, hotel, car-hire or fuel pre-auth	Expire by MCC and scheme rules, release the hold, notify the customer. A clearing that arrives after expiry must still post, since the issuer is liable.
4. Remainder released on partial settlement (applySettlement releases the whole hold)	Final amount below the estimate, or a split shipment with more clearings to come	Release only when the clearing is marked final. For multi-clearing merchants, keep the remainder until the final flag or expiry.
5. Refused, left APPROVED: SETTLEMENT_EXCEEDS_HOLD	Tip added after auth, fuel top-up, incremental hotel charges	Allow over-clearing up to the MCC tolerance; support incremental auths. Refusing turns it into case 3 while the clearing still arrives.
6. Second settlement refused: AUTH_NOT_OPEN on a settled auth	Split shipment, or a genuine duplicate clearing	Allow multiple clearings when flagged. Reject and alert true duplicates, keyed on the scheme transaction ID, not the auth ID.
7. Settlement against a declined auth refused: AUTH_NOT_OPEN	Merchant forced the sale offline	Post it (the scheme settles regardless) and open a chargeback case. Refusing hides money that has moved.
8. Wrong account refused: AUTH_ACCOUNT_MISMATCH	Card reissued or re-linked between auth and clearing; routing error	Resolve through card→account link history; send anything unresolved to suspense with an exception case.
9. Cannot be cancelled: REVERSAL accepts only CREDIT/DEBIT	Merchant void, till cancellation, network timeout reversal (0400)	Required before go-live: an idempotent auth reversal that releases the hold immediately, matched on scheme identifiers.

A settlement that matches no auth at all (AUTH_NOT_FOUND, a force post) follows the reasoning of case 7; REJECTED.md explains why this core still refuses it.

4. What I cut and why

Each cut kept the core small and exactly aligned to the brief. Each one defers a named production risk.

Cut	Why out of scope	Production risk deferred
Double-entry	Brief asks for account balances only	No contra legs (fee income, interest expense, scheme settlement), so the ledger cannot reconcile to the GL.
Persistence and idempotency	"No persistence"	A crash loses everything. ingest does not check event IDs, so a replayed credit books twice (verified); only auth IDs are deduplicated.
Concurrency control	Single-threaded replay	Check-then-hold race across processes: two auths spend the same funds. Needs per-account serialization.
Business calendar	Six abstract days	No weekends, UAE holidays, cut-offs or time zone; value dates on non-business days; day-count errors in interest.
Runtime input validation	Inputs are a fixed in-code array	Types are compile-time only. A numeric amount throws a <code>TypeError</code> that ingest does not catch, so one bad message crashes the replay (verified).
Product depth	One fee, one rate	No overdraft limit (settlements and backdated debits go arbitrarily negative), no fee caps; conventional interest only, unusable for Islamic products.
VAT and FX	Not in the brief; no rates given	Fee VAT neither charged nor credited on refund; a negative BHD account is never charged (<code>NO_FEE_SCHEDULE</code>); cross-currency auths rejected.
Auth lifecycle depth	Stream needs only approve, decline and settle	§3 cases 3–9: funds locked until manual action, customer-visible declines, disagreement with scheme settlement.
Reversal scope	Only E9 needed it	No partial reversal, settlement reversal or refund link; reversing a settlement would leave its auth <code>SETTLED</code> .
Unapproved fee refunds	Keeps the fee rule true of the final ledger	Money out with no maker-checker; a backdated credit becomes a fee-refund path (§2).
Audit metadata	Not needed to replay	No user, timestamp, channel or reason on events; cannot answer "who posted this and why".
Account lifecycle	Two fixed accounts	No open, close, freeze, dormancy or KYC status; postings to frozen accounts go through.
Period close and memory	Six-day window	§1: unbounded lookback, quadratic report memory, restated closed periods.
Property and fuzz tests	Time	Order-dependence bugs (the known failing test) are found by reading, not by the suite. Shuffling same-day events would catch them mechanically.